

HDL Final Project Report

Group Number: 30

Members: 112062130 侯佑勳
112062326 孔祥光

Contents:

Introduction	1
Motivation	1
System Specs & Testing Results	1
1. Balancer Mechanism	2
2. Servomotor Module	3
3. Resistive Touchscreen with 4 Wires	4
4. PID controller module	8
Conclusion	11
Possible Improvements	11
Reference	12

Introduction

The primary goal of this project is to design and implement a ball-balancing mechanism using a Basys3 FPGA board, servo motors, and a 4-wire resistive touchscreen. The system applies PID control theory to maintain balance and achieve stable positioning of the ball on a flat surface.

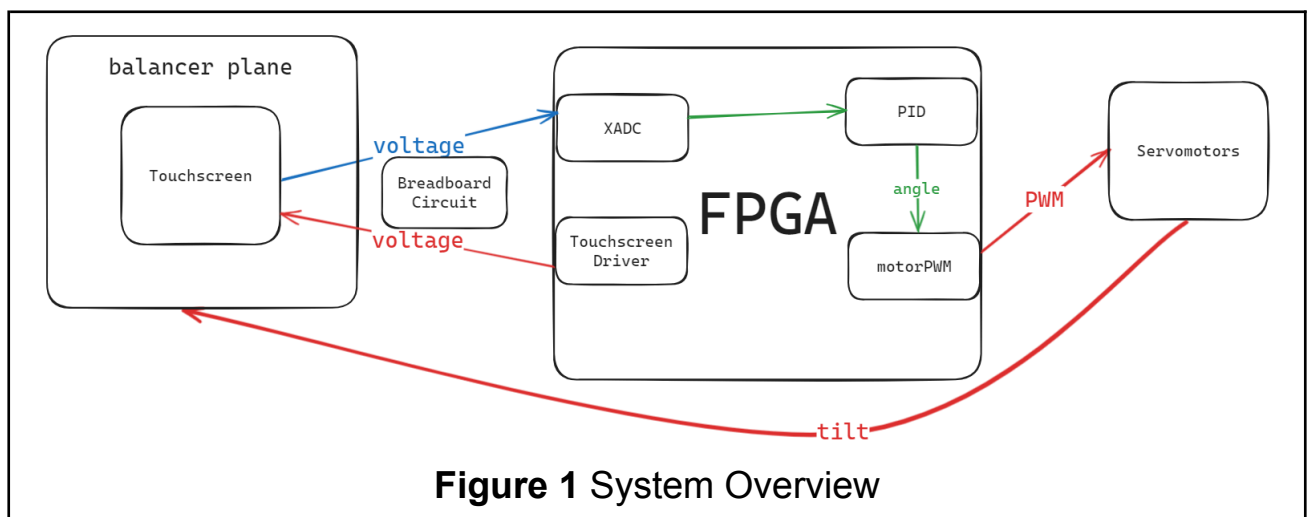
Motivation

We were first inspired by the experiment done by Poland students, at <https://www.youtube.com/watch?v=F8R91bAOHSU>. Since we have some background knowledge about PID and want to learn how to properly apply a touchpad, we start to conduct this final project.

Balancing mechanisms have wide applications in robotics, mechatronics, and control systems. Developing a functional model allowed us to explore control theory in a practical setting, improve our hardware designing skills, and integrate analog-digital interfacing.

System Specs & Testing Results

The system includes a flat plane controlled by two servomotors to adjust tilt angles for ball balancing. It obtains the coordinates of the ball on a resistive touchscreen. The PID controller calculates the required adjustments based on touchscreen input. All of the calculations and signal drives are done with the FPGA.



1. Balancer Mechanism

Design

By referencing various experimental videos on YouTube, we designed our own balancer mechanism.

For the designing part, we use Inventor to complete a CAD and do some simulation. First we should ensure that the side link's movement is straight up and straight down, which is important for calculating the relationship between the plane angle(α) and the servo arm angle(θ).

$$\sin(\alpha(t)) = 2h/L_{\text{plane}}, \sin(\theta(t)) = h/r_{\text{arm}},$$

which imply $\sin(\alpha(t)) = 2\sin(\theta(t))r_{\text{arm}}/L_{\text{plane}}$

Second, to ensure the x-axis and y-axis motions are independent, the side support link's location needs to be at the middle of the plane side, because it's the plane's center of gravity, and for symmetry purposes.

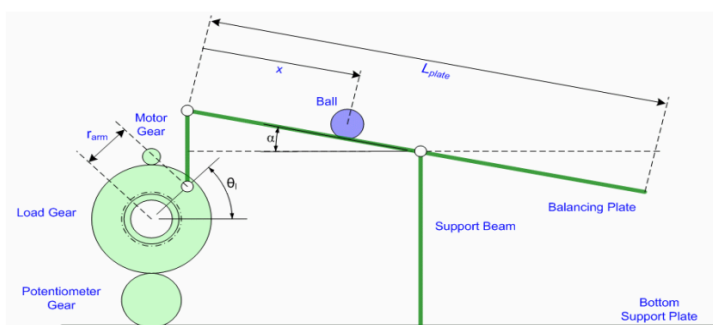


Figure 6: Schematic showing the x-direction of the 2D Ball Balancer system

Figure 2 Schematic for the x-direction of the Ball Balancer
(made-for-science-Quanser-2-dof-ball-balancer-UserManual)

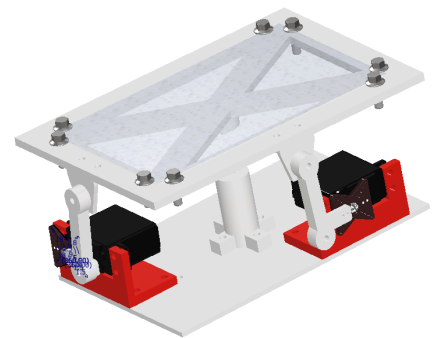


Figure 3 Our Mechanism

For the mechanical part, we use a universal joint on the middle support pillar to allow the system to move freely at multiple angles. In addition, for the side link, we use self-lubricating spherical bearings to ensure the independence of x-axis and y-axis motions and the link's flexibility within a certain range.



Figure 4-1 universal joint & spherical bearing

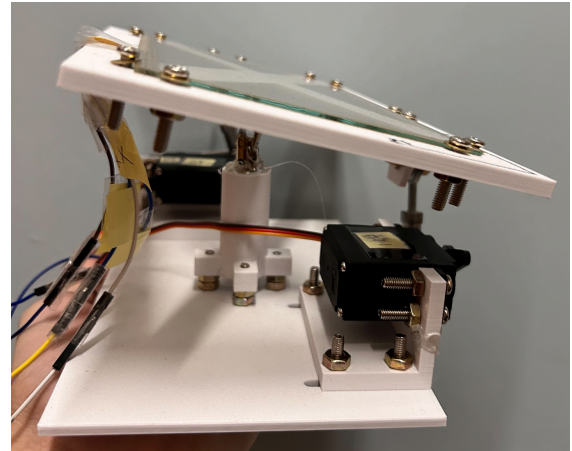


Figure 4-2 Ball Balancer Side View

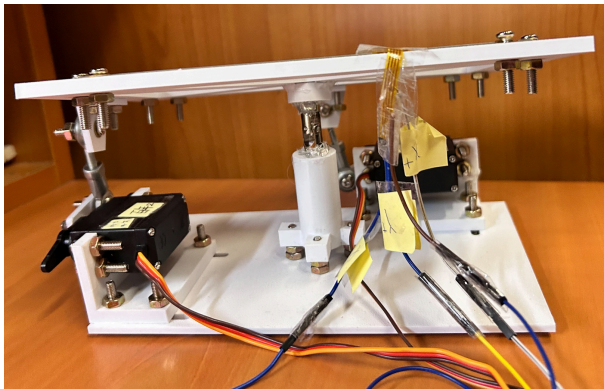


Figure 4-3 Ball Balancer Side View

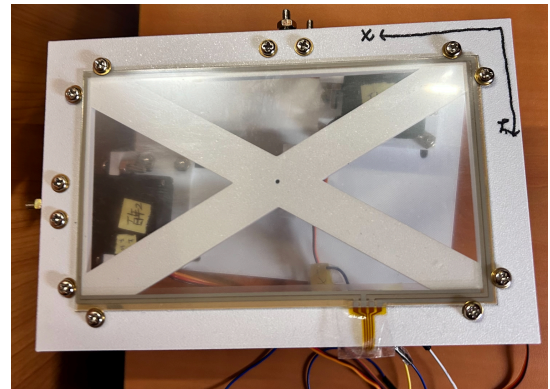


Figure 4-4 Ball Balancer Top View

Testing

[In Video 1](#) we set different angle to the servo and whole system to test whether x-axis and y-axis movements are independent.

2. Servomotor Module

- **Model:** MG995
- 3-wire control: ground, power & PWM (black, red and orange)
- Rotation controlled via a 50Hz PWM signal

- Duty cycle ranges: 2.5ms (0°) to 12.5ms (180°)
- Powered directly by a battery pack (4.8V ~ 7.2V), and the ground need to be synced with the GND of FPGA



Design

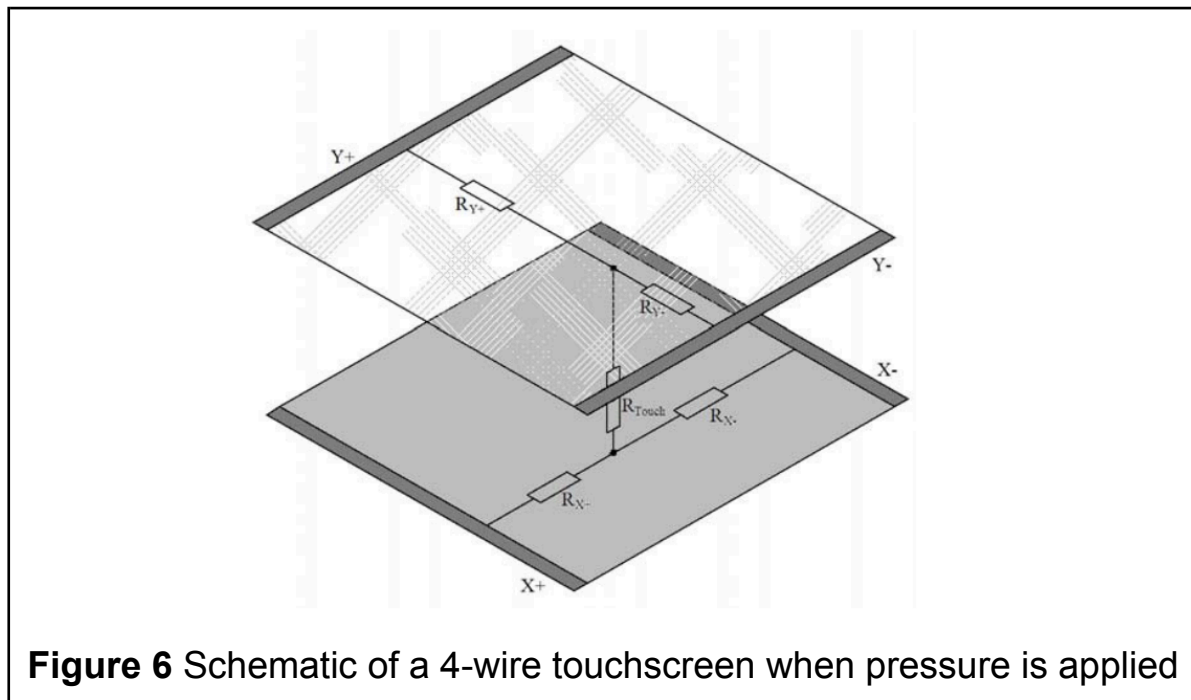
We control the servomotor's rotation by changing the duty cycle, namely, the duration of PWM, which is achieved by a PWM generator module, shown in the figure below.

Testing

[In Video 2](#) we test the specified duty cycle will cause the motor to change to a distinct angle. We set the balancing position of servo motor to 60 degrees and the angle range is between 0~120 degree(if the angle is >120 degrees, the link's won't move straightly which cause less effect on plane's rotation)

3. Resistive Touchscreen with 4 Wires

Design



To measure the x-y coordinate of the ball on the panel, the voltage gradient must be applied once in vertical and then in horizontal direction: first a supply voltage must be applied to one layer and a measurement of the voltage across the other layer is performed, next the supply is instead connected to the other layer and the opposite layer voltage is measured. A finite state machine is employed to alternate between X and Y coordinate reading states.

	X+ Excite	X- Excite	Y+ Excite	Y- Excite
Idle	Gnd	Hi-Z	Hi-Z	Pull up / Int
Read_X	Gnd	Vcc	Hi-Z	ADC
Read_Y	Hi-Z	ADC	Gnd	Vcc

Table 1 Connection of the 4 wires in reading different directions

Since the output from the touchscreen is an analog signal of voltage, we use the built-in XADC component of Basys 3 and the XADC wizard IP provided by Xilinx to access the voltage reading on the FPGA. Note that the JXADC ports can only read voltage up to 1V, so the

touchscreen is interfaced through a voltage divider circuit to ensure XADC input compatibility.

To achieve reading x and y coordinates in different cycles, we use NPN transistors to design a circuit to efficiently exchange signals to the 4-wire touchscreen and FPGA.

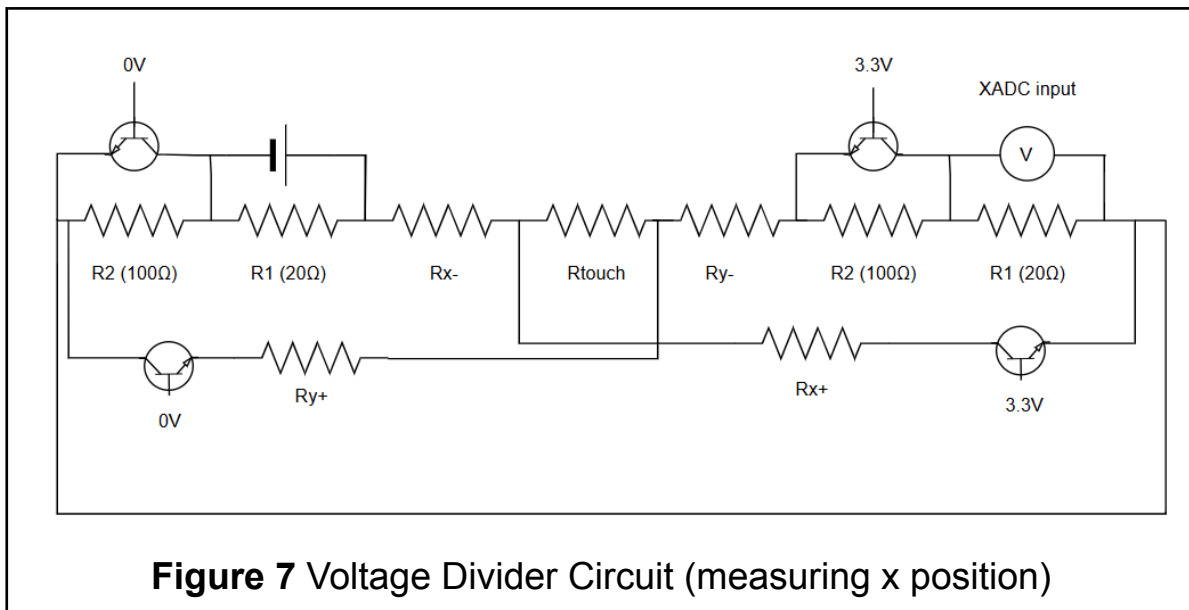
We have tried lots of methods, and succeed to let the input and output just on the same wire, same position on the circuit.

As **Table 1** above shows, X- and Y- ports on both FPGA board and touchscreen change between Vcc and ADC. So for the ADC state, we give Base of the transistor zero signal. The current will flow through R1 and R2, then reduce voltage successfully. For the Vcc state, we give Base of the transistor signal one, the E and C will be connected, so the current won't flow through the step-down circuit.

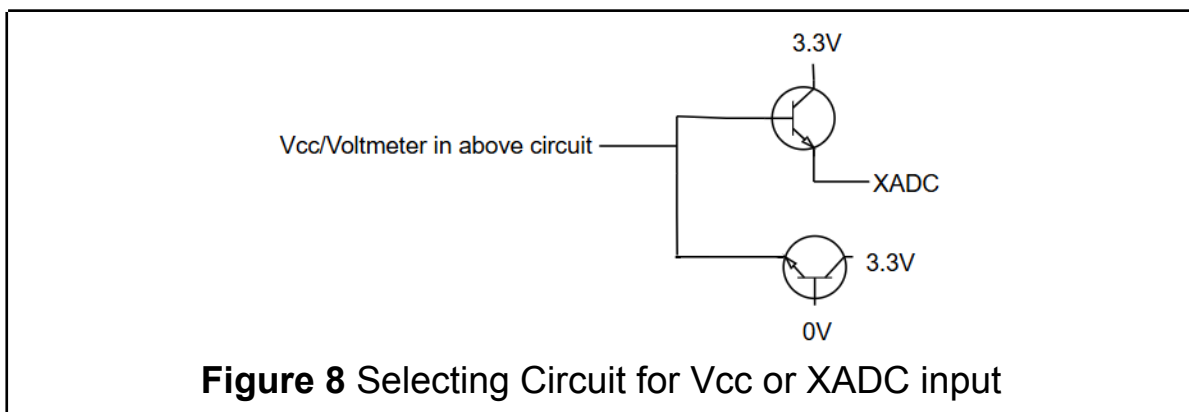
Let's discuss the X+ and Y+ ports on the touchscreen. It should be HighZ when its axis is being measured, and Ground when the other axis is being measured. We use one transistor to achieve that. We give C ground signal and the Base a logic high signal, the E and C will be connected, then X+ will be successfully grounded. As for HighZ, we just break the connection between C and E by giving the base logic low signal. In addition, we simply assign X+ and Y+ ground as a negative port's standard.

However, we later found out the inout signal cannot use the same port on the FPGA. Basys 3 JXADC ports support either analog input **or** digital input-output, but we need to do analog input **and** digital output.

So we tried using two transistors as switches to separately control input and output. Sadly, this method also failed because if the Base is connected to 3.3V, and the touch screen is not being touched, meaning no voltage through C, E will still be high.



Finally we revised the transistor circuit by using the signal attenuation. We add two more transistors, one for Vcc switch, the other for ADC signal input.



Testing

[In Video 3](#), we test the output voltage of the x-direction of the touch panel. After flowing through a reduced voltage circuit, the max output is 0.5v, the min is 0v, but there are still some noises.

The below picture is the real circuit realized by breadboard and jumper wires. It still suffers heavily from noise issues.

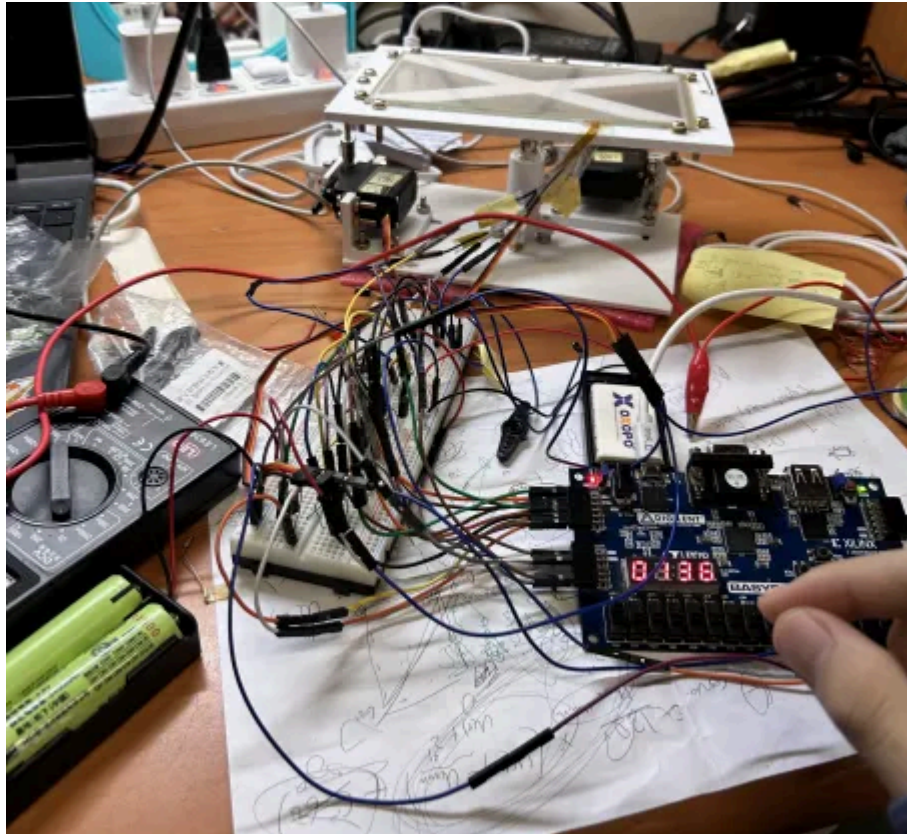


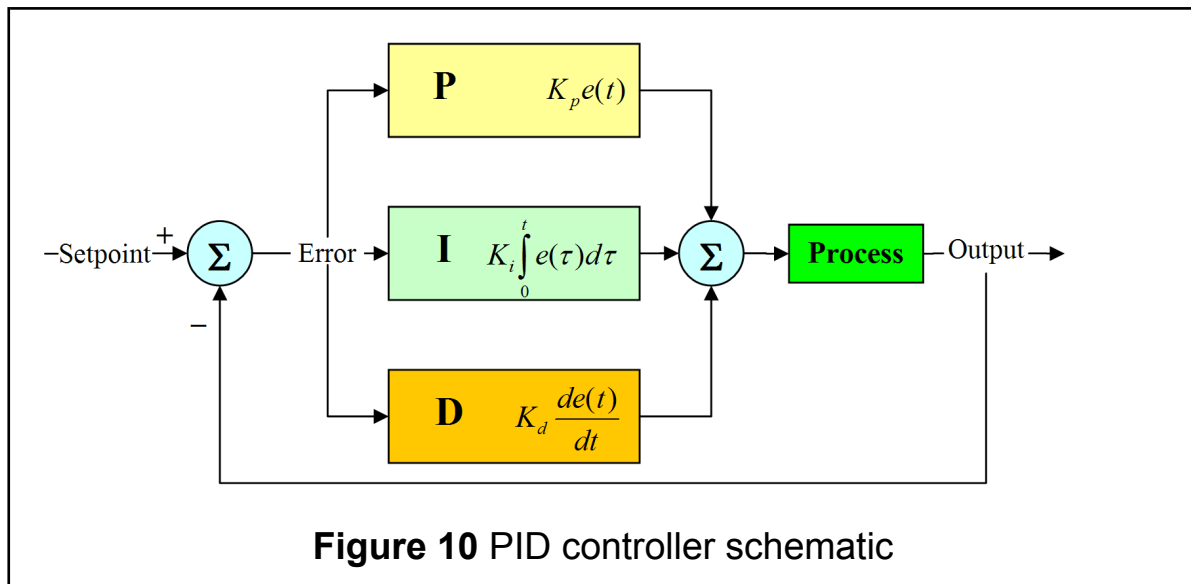
Figure 9 Breadboard Circuit to manage I/O of touchscreen

4. PID controller module

Input: Setpoint[9:0], Process Value[9:0], Kp[15:0], Ki[15:0], Kd[15:0]

Output: out[7:0] (0~120 degree)

We use a closed loop Proportional-Integral-Derivative (PID) controller to connect the touch panel's data and the duty cycle for servomotors. In the PID module, the unit of the input is the 10-bit voltage reading, and that of the output is degree.



Design

The integral and derivative are implemented by summation and subtraction, as the clock cycle is very fast (10ns), this approximation is acceptable. Here is how they are calculated.

Proportional term (Error) = (Setpoint - Process Value)

Integral term = Integral term + Error

Derivative term = Error - PrevError

Total error gain = $K_p * P + K_i * I + K_d * D$

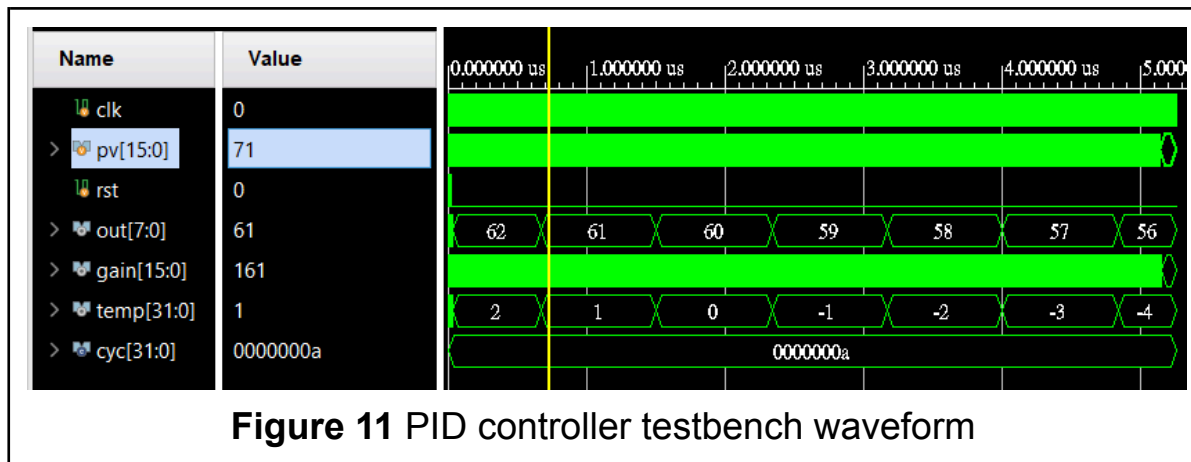
The total gain indicates how much the plane should tilt, the larger the value, the bigger the degree to turn. Therefore, we use linear interpolation to produce output. The range of total error gain is MIN_GAIN and MAX_GAIN, determined by empirical observations, and that of Output is 0 ~ 120 degree. Here is the equation of how we produce the output.

Output = $(\text{total error gain} - \text{MIN_GAIN}) * (\text{DEG_MAX} - \text{DEG_MIN}) / (\text{MAX_GAIN} - \text{MIN_GAIN}) + 60$

(add 60 to turn the result from [-60,60] to [0,120] degrees)

Testing

Because in the PID module, we use signed decimals to calculate, so we also design the testbench to ensure all the data is calculated correctly.

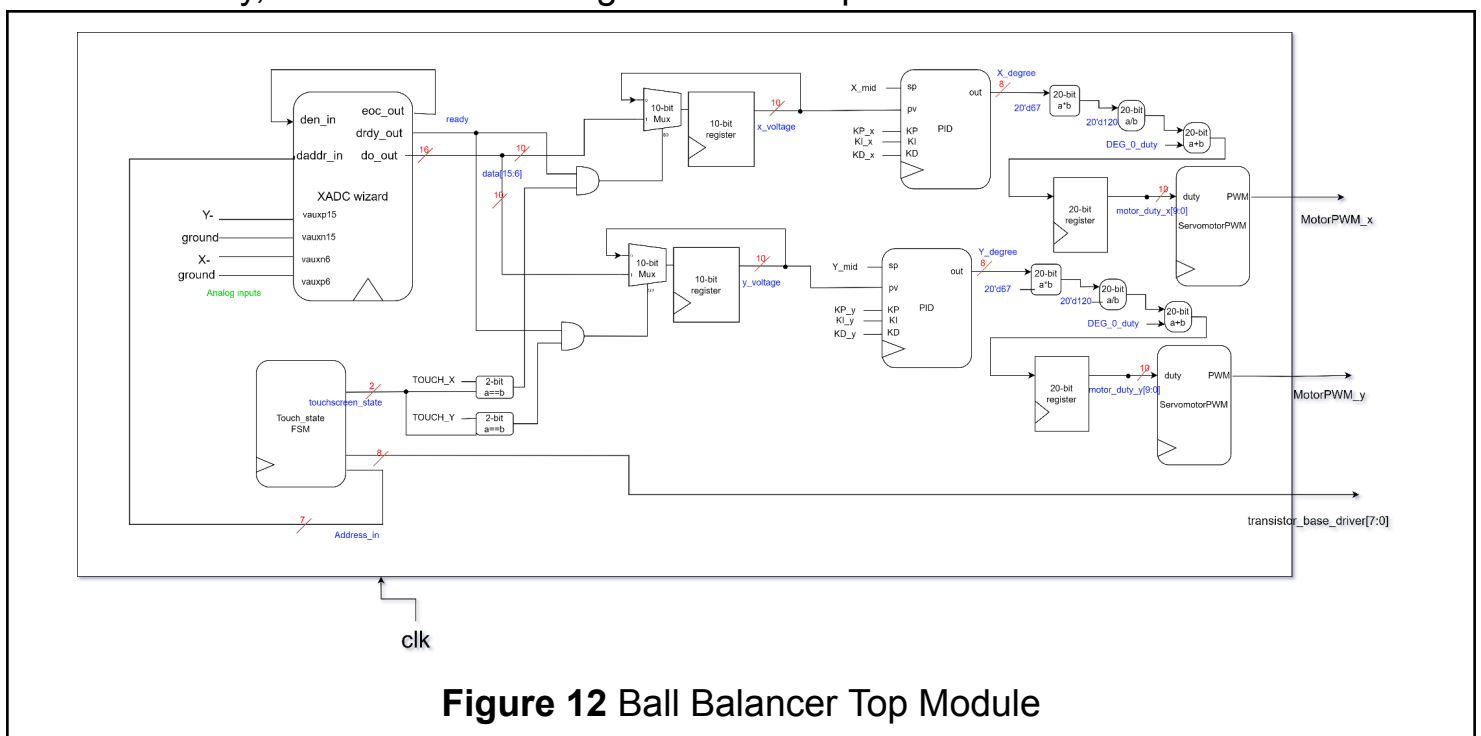


As we can see, the output and the total gain is in a linear relation, and when the error gain is near zero, the output rotation angle is ~60 degrees, which is the balance position.

For our project, we found that using PD control is enough.

[Video 4-1](#) shows a P controller, and [Video 4-2](#) shows a PD controller. We can observe the plane turning more smoothly with the derivative term applied, since it predicts future errors..

Finally, here's the block diagram for our top module.



Conclusion

In this project, we successfully used the touch screen as input to know the object's x and y position, and used the PID control theorem to control the servomotors.

For the result, we can control not scrolling the object's balance and just change input or output signals instead of changing the real circuit to read x and y position in turn. We also met some problems while conducting experiments. The touchscreen inputs are rather unstable, even after sample averaging, the interfacing between our FPGA and the hardware components relies heavily on jumper wires. And when the plane starts moving, it may cause the wires to fall off a bit, resulting in faulty communication.

Overall, the project demonstrated an implementation of a ball-balancing mechanism using FPGA-based control. The integration of analog inputs with digital processing highlights the versatility of HDL in real-world applications. While achieving stability posed challenges, iterative improvements to the hardware and control logic should resolve most issues.

Possible Improvements

- Replace the 4-wire resistive panel with a 5-wire panel to simplify I/O management, as the 5-wire panel has a wire specifically dedicated to output, therefore eliminating the need for inout hassle and the mess on the breadboard circuit.
- Adopt some anti-noise measures, e.g. add buffer to our breadboard circuit.
- Apply more fine-tuning for the PD controller
- Introduce automatic parameter tuning for the PID controller to minimize manual effort.

Reference

- [ATMEL, AVR341: Four and five-wire Touch Screen Controller](#)
- [7 Series FPGAs and Zynq-7000 SoC XADC Dual 12-Bit 1 MSPS Analog-to-Digital Converter User Guide \(UG480\)](#)
- [Basys 3 XADC Demo](#)
- XADC Wizard v3.3 LogiCORE IP Product Guide (PG091)
- made-for-science-Quanser-2-dof-ball-balancer-UserManual
- [2D Ball Balancer Control using QUARC from Naval Postgraduate School](#)